LM -> Language Model
=> understands and predict text
types of lm
-> traditional lm
-> neural lm

LLM -> large language model
LLM = predicts next word → repeatedly → forms intelligent output
foundation model -> llm
more parameter -> more complex the model
llm -> data+architechture+training
                |
            transformer

pretraining => RLHF (reinforcement learning with human feedback)

fine tuning

GPT => Generativ Pre-trained Transformer
transformer:
input => tokens => each token is asociated with vectors => attention block(to talk to each other) => multilayer perceptron(long list of question) => it repeats the attention and multilayer

tensor(input) => tunable parameter(weights)=>outputs
weights(brains-what defines the model) -> blue nd red .. data(what the model processes) -> grayscale

embedding matrix(We) => associate each token with high dimentional vector words vector(context size)
unembedding matrix
import gensim.dowloader
model =m gensim.downloader.load("glove-wiki-gigaword-50")
model["tower"]

softmax=> converts into 0 to 1 range
tempature -> t is put into the softmax 2 is defauly
logits ----> probabilitites

attention:
key and query
masking applying neg infinity for normalization
the value matrix

Neural network:
neuron -. thing tht holds a number(fuction)
28 x 28 = 784 neuron each of these hold grayscale numbers
the number inside the neuron is called activation thses make the first layer of the neural network

last layer => how much the system thinks the data corresponds with the data

hidden layer = ?

w1a1+w2a2+.....wnan
sigmoid -> used
784 x 16 each
784x16+16x16+16x10 ->weights
16+16+10 -> biases
total = 13002

learning => fiding the right weights and bias

Gradient descent: algorithm

increase b
increase wi in proportion to ai
change ai in proportion to wi

hebbian theory: neurons tht fire together wire together

mini batches

PROMPT ENGINEERING:

programming in natural language
task,role,format,constraints
wiperflow - voice to txt

commanding vs steering
-> to be specific & set the scene
   role,audience,tone,format

few shot prompting

Vector Database:
 semantic gap =>
 vector embedding : arry of numbers
 embedding model -> clip(images) glove(txt)

vector indexing -> ann algorithms
HNSW IVF
core feature of rag
-retrieval
-augmentation
-generation

1.retrieval
semantic searching: meaning and the context of the query is used to match the document
2.augmentaion: the retrieved data is injected into the prompt at run time
3.generation: generates the response from tha above two steps

chucking stratery - size of the chuch and overlap
embedding stratery - which embedding model use
retrival stratery - control the thershold how the words similar

rag:
chucks -> embedding model -> vector database
input prompt + chucks => context window
content injection: rag

long context : we remove embedding and database and include the document in the context window

fine-tuning llm:training a pre-trained llm to our specific need
when:
-consistent formatting or style
-domain space data
-reduce cost

unsolth - used for fine tuning
10web api
=> 1 gathering data eg prompt,eg output
=> 2

zapier MCP server -> free opensource

LANGCHAIN:

LangChain = a toolkit to build apps using LLMs
it helps you connect everything together:
LLM (like Gemini or GPT)
your data (PDF, DB, files)
logic (prompts, steps, memory)
Input → Prompt → LLM → Output
. LLM / Chat Model
Talks to AI
2. Prompt
Controls how you ask
3. Chains
Connect multiple steps
4. Retriever
Fetch data (used in RAG)
5. Memory
Stores chat history

there document in langchain
content + metadata => document

```
[
  Document(page_content="chunk1", metadata={"page": 1}),
  Document(page_content="chunk2", metadata={"page": 2})
]
```

like this

Hugging_face models
how to use them:
+> pipeline

```
pip install transformers
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("AI is", max_length=20)
print(result)
```

=> Slightly advanced — Load model + tokenizer

Use this when you want more control.

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "gpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

inputs = tokenizer("AI is", return_tensors="pt")
outputs = model.generate(**inputs, max_length=20)

print(tokenizer.decode(outputs[0]))
```

=>Using Hugging Face API

```
pip install huggingface_hub
from huggingface_hub import InferenceClient

client = InferenceClient(model="gpt2")

result = client.text_generation("AI is", max_new_tokens=20)
print(result)
```

Model → brain (GPT, BERT, etc.)
Tokenizer → converts text → numbers
Pipeline → shortcut (model + tokenizer together)

Types of models in Hugging Face
1. Text Generation (LLMs)

   Used for chat, writing, Q&A

Examples:

GPT-2 → old, weak
Mistral 7B → strong, efficient
LLaMA 2 → popular
Falcon 7B → good instruct model

   Use this for: chatbot, RAG, assistants

2. Text Classification

   Classify text

Examples:

spam detection
sentiment analysis

Model:

BERT
3. Summarization

   Convert long text → short

Example model:

BART
4. Translation

   Language conversion

Example:

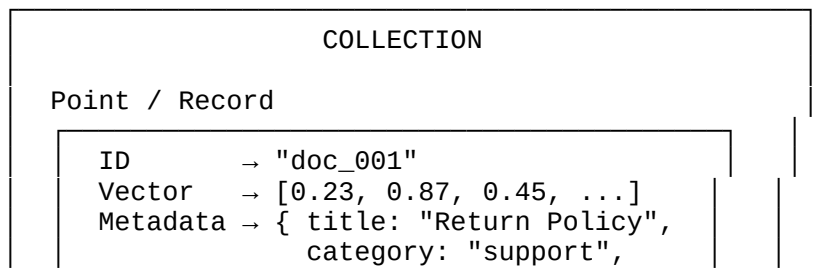
T5
5. Image models

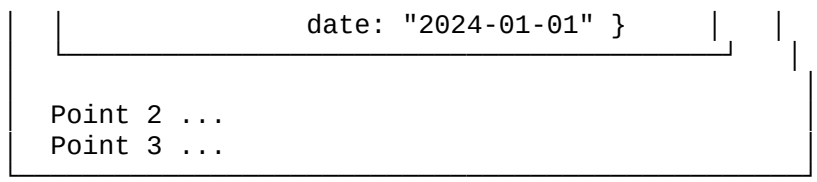
   Computer vision

Examples:

Stable Diffusion → generate images
ResNet

Collections :





Vector Dimension
Must match your embedding model's output size.

```
python# OpenAI text-embedding-ada-002 → 1536 dimensions
# SBERT → 768 dimensions
# Custom model → whatever it outputs

collection = {
  "name": "support_docs",
  "dimension": 1536
}
```

2. Distance / Similarity Metric
How vectors are compared inside this collection.

```
python"metric": "cosine"    # Best for text
"metric": "euclidean" # Best for image/spatial
"metric": "dot"       # When vectors are normalized
```

3. Index Type
Algorithm used for fast search.

```
python"index": "hnsw"   # Most common, graph-based
"index": "ivf"    # Cluster-based, good for huge datasets
"index": "flat"   # Brute force, only for small sets
```